

Solution of Biharmonic Equation in Complicated Geometries with Physics Informed Extreme Learning Machine

Summary of Dwivedi and Srinivasan, Journal of Computing and Information Science in Engineering 202

1 Introduction

The biharmonic equation is a fourth-order partial differential equation that appears in numerous applications across solid and fluid mechanics. In two dimensions, the biharmonic equation takes the form:

$$\nabla^4 u(x, y) = \frac{\partial^4 u}{\partial x^4} + 2 \frac{\partial^4 u}{\partial x^2 \partial y^2} + \frac{\partial^4 u}{\partial y^4} = R(x, y), \quad (x, y) \in \Omega \quad (1)$$

with Dirichlet boundary conditions:

$$u = G_1(x, y), \quad \frac{\partial u}{\partial n} = G_2(x, y), \quad (x, y) \in \partial\Omega \quad (2)$$

where Ω is a bounded open set in $\mathbb{R}^2$ with a smooth boundary $\partial\Omega$, and $\partial u / \partial n$ is the normal derivative of u on $\partial\Omega$.

This equation is central to modeling various physical phenomena including:

- Slow 2D Newtonian viscous fluid flows
- Geophysical flows in the Earth's mantle
- Roll coating and polymer processing
- Stress functions in 2D elastostatic problems
- Vertical displacement in thin clamped plates under external loads

Traditional numerical methods for solving the biharmonic equation include the Finite Element Method (FEM) and Finite Difference Method (FDM). However, these mesh-based approaches face significant challenges when dealing with irregular domains, as mesh generation becomes either impossible or prohibitively expensive.

Physics Informed Neural Networks (PINNs) offer a more flexible framework for solving PDEs in complex geometries but suffer from slow training speeds, optimization difficulties (vanishing gradients, local minima), and the need for hand-tuning of hyperparameters. The Physics Informed Extreme Learning Machine (PIELM) addresses these limitations by combining the physics-informed loss function approach of PINNs with the computational efficiency of Extreme Learning Machines.

2 Physics Informed Extreme Learning Machine (PIELM) for Biharmonic Equations

2.1 PIELM Architecture and Algorithm

PIELM employs a single-layer feedforward neural network with randomly assigned input layer weights that remain fixed (non-trainable). Only the output layer weights are determined analytically using the least-squares method. This approach significantly reduces computational time compared to traditional gradient-based optimization methods.

For a PIELM with N^* neurons in the hidden layer, if we define $\vec{\chi} = [x, y, 1]^T$, $\vec{m} = [m_1, m_2, \dots, m_{N^*}]^T$, $\vec{n} = [n_1, n_2, \dots, n_{N^*}]^T$, $\vec{b} = [b_1, b_2, \dots, b_{N^*}]^T$, and $\vec{c} = [c_1, c_2, \dots, c_{N^*}]^T$, then the output of the k^{th} hidden neuron is:

$$h_k = \phi(z_k) \quad (3)$$

where $z_k = [m_k, n_k, b_k]\vec{\chi}$ and $\phi = \tanh$ is the nonlinear activation function. The PIELM output is given by:

$$f(\vec{\chi}) = \vec{h}\vec{c} \quad (4)$$

The partial derivatives needed for the biharmonic equation are calculated exactly (without discretization error) as:

$$\frac{\partial^p f_k}{\partial x^p} = m_k^p \frac{\partial^p \phi}{\partial z^p} \quad (5)$$

$$\frac{\partial^p f_k}{\partial y^p} = n_k^p \frac{\partial^p \phi}{\partial z^p} \quad (6)$$

2.2 Physics-Based Error Incorporation

PIELM incorporates the physics of the problem by introducing physics-based training errors ξ_R , ξ_{G1} , and ξ_{G2} . These errors correspond to the PDE residual and boundary condition violations:

$$\xi_R = \frac{\partial^4 f}{\partial x^4} + 2\frac{\partial^4 f}{\partial x^2 \partial y^2} + \frac{\partial^4 f}{\partial y^4} - R, \quad (x, y) \in \Omega \quad (7)$$

$$\xi_{G1} = f - G_1, \quad (x, y) \in \partial\Omega \quad (8)$$

$$\xi_{G2} = \frac{\partial f}{\partial n} - G_2, \quad (x, y) \in \partial\Omega \quad (9)$$

For a PIELM that can solve the biharmonic equation with zero error, these error terms must be zero:

$$\xi_R = \vec{0} \quad (10)$$

$$\xi_{G1} = \vec{0} \quad (11)$$

$$\xi_{G2} = \vec{0} \quad (12)$$

These constraints lead to a system of linear equations of the form $A\vec{c} = \vec{b}$, which can be solved using the Moore-Penrose generalized inverse:

$$\vec{c} = \text{pinv}(A)\vec{b} \quad (13)$$

2.3 Detailed PIELM Equations for Biharmonic Equation

If $\vec{x}_f, \vec{y}_f$ denote collocation points vectors, $\vec{x}_{bc}, \vec{y}_{bc}$ denote boundary points vectors, $\vec{I}$ denotes the bias vector, and $\odot$ refers to the Hadamard product, then with $X_f = [\vec{x}_f, \vec{y}_f, \vec{I}]$, $X_{bc} = [\vec{x}_{bc}, \vec{y}_{bc}, \vec{I}]$, and $W = [\vec{m}, \vec{n}, \vec{b}]$, the PIELM equations for the biharmonic problem are:

$$\phi^{(4)}(X_f W^T) \odot \vec{c} \odot (\vec{m} \odot \vec{m} \odot \vec{m} \odot \vec{m} + 2\vec{m} \odot \vec{m} \odot \vec{n} \odot \vec{n} + \vec{n} \odot \vec{n} \odot \vec{n} \odot \vec{n}) = R(\vec{x}_f, \vec{y}_f) \quad (14)$$

$$\phi(X_{bc} W^T) \vec{c} = G_1(\vec{x}_{bc}, \vec{y}_{bc}) \quad (15)$$

$$\phi'(X_{bc} W^T) \odot \vec{c} \odot \vec{m} = \frac{\partial}{\partial x} G_2(\vec{x}_{bc}, \vec{y}_{bc}) \quad (16)$$

$$\phi'(X_{bc} W^T) \odot \vec{c} \odot \vec{n} = \frac{\partial}{\partial y} G_2(\vec{x}_{bc}, \vec{y}_{bc}) \quad (17)$$

2.4 PIELM Implementation Steps

The key steps in implementing PIELM for the biharmonic equation are:

Algorithm 1 PIELM for Biharmonic Equation

- 1: Assign the input layer weights $\vec{m}$, $\vec{n}$, and $\vec{b}$ randomly
 - 2: Define expressions for ξ_R , ξ_{G1} , and ξ_{G2} based on the PDE and boundary conditions
 - 3: Assemble the three sets of equations in the form of $A\vec{c} = \vec{b}$
 - 4: Calculate the output layer weight vector using $\vec{c} = \text{pinv}(A)\vec{b}$
-

An important advantage of PIELM is that it provides a criterion for selecting the number of neurons in the hidden layer. The optimal number of hidden units equals the number of constraints: $N^* = N_R + N_{G1} + N_{G2}$, where N_R , N_{G1} , and N_{G2} represent the number of collocation points and boundary condition points. For lower neuron counts, solutions become unstable, while higher neuron counts do not significantly improve accuracy.

3 Numerical Examples and Results

The paper presents three categories of numerical tests to evaluate PIELM performance:

3.1 Comparison with PINN on Regular and Irregular Domains

Two test cases were considered using the method of manufactured solutions:

1. Circular domain: $\partial\Omega_1 : x^2 + y^2 = \frac{1}{4}$ with exact solution $u(x, y) = x^2 + y^2 + e^x$
2. Irregular star-shaped domain: $\partial\Omega_2 : \frac{x}{y} = \{\frac{3}{5} + \frac{1}{4} \sin(2\theta)\} \{\cos(\theta), \sin(\theta)\}$, $\theta \in [0, 2\pi]$ with exact solution $u(x, y) = x^2 + y^2 + e^x \cos(y)$

Using just 150 training data points, PIELM achieved an accuracy of order 10^{-6} for both domains with computation times on the order of milliseconds. In contrast, a PINN with two hidden layers (10 neurons each) required 8-10 seconds to reach the same accuracy level, making PIELM at least 1000 times faster.

The study also demonstrated that PIELM exhibits excellent generalization properties, accurately predicting the solution for 5000 new data points that were different from the training data. This indicates that PIELM does not suffer from overfitting and is less susceptible to the vanishing gradient problem that affects deeper networks.

3.2 Lid-Driven Cavity Problem: Comparison with FDM

The biharmonic equation was applied to the Stokes flow in a square lid-driven cavity. The steady, incompressible flow problem takes the form:

$$\nabla^4 \psi = 0 \quad (18)$$

with appropriate boundary conditions for the stream function ψ .

Three cases were examined with different top and bottom lid velocities:

1. $u_B = 0, u_T = 1$: Flow consists of a large, centrally symmetric eddy
2. $u_B = -1, u_T = 1$: Flow shows a large circulatory motion with two eddy centers and a saddle point at the center
3. $u_B = 1, u_T = 2$: Flow exhibits two large eddies in the cavity

PIELM successfully captured all three flow structures using just 20×20 uniformly spaced data points with a computation time of approximately 0.5 seconds. However, for rectangular geometries, a standard FDM solver converged to the solution in just 1 second compared to 10-11 seconds for PIELM with a 40×40 uniformly spaced dataset. This indicates that FDM is faster than PIELM for regular domains.

Due to the discontinuous boundary conditions at the lid corners, the PIELM solution showed maximum errors of order 10^{-3} and 10^{-2} along the horizontal and vertical directions, respectively.

3.3 Biharmonic Equation on a Complex Domain: Map of Australia

To demonstrate PIELM's advantage for complex geometries, the biharmonic equation was solved on a scaled map of Australia. Standard mesh generation tools in MATLAB and COMSOL failed to create a mesh for this complex geometry, highlighting the limitations of conventional methods.

Using 711 training and 1661 testing sample points, PIELM achieved a solution with a maximum error of order 10^{-6} and a total CPU time on the order of 10^{-1} seconds. This demonstrates PIELM's ability to handle complex domains where mesh-based methods are impractical.

3.4 Effect of Network Size on PIELM Performance

The study examined the relationship between generalization performance and network size using a modified test case with a sharper gradient solution $u(x, y) = x^2 + y^2 + e^{3x}$. Results showed that PIELM performance remains stable across a wide range of hidden node counts, though it deteriorates with very few neurons.

Setting the number of hidden units equal to the number of constraints ($N^* = N_R + N_{G1} + N_{G2}$) provides optimal results, as performance does not significantly improve with additional neurons. This feature of PIELM reduces the arbitrariness in neural network architecture selection that affects traditional deep learning approaches.

4 Comparison of Computational Methods

The paper provides a systematic comparison between mesh-based methods, PINN, and PIELM across several dimensions:

4.1 Basis Functions

- Mesh-based methods: Piecewise polynomials (typically linear)
- PINN: Deep neural networks (nonlinear)
- PIELM: Shallow neural networks (nonlinear)

4.2 Unknowns

- Mesh-based methods: Solution values at mesh points
- PINN: All weights and biases
- PIELM: Only output layer weights

4.3 Physics Awareness

- Mesh-based methods: Through appropriate discretization schemes
- PINN: Through physics-based cost functions
- PIELM: Through physics-based errors

4.4 Solution Methods

- Mesh-based methods: Iterative solution of algebraic equations
- PINN: Iterative minimization using backpropagation
- PIELM: Direct calculation through matrix inversion

4.5 Sources of Error

- Mesh-based methods: Discretization errors, numerical instability, mesh generation issues
- PINN: Vanishing gradients, local minima, optimization errors
- PIELM: Ill-conditioned coefficient matrices

4.6 Computational Performance

- Regular domains: Mesh-based methods $\lesssim$ PIELM $\lesssim$ PINN (in terms of speed)
- Complicated domains: PIELM $\lesssim$ Mesh-based methods $\lesssim$ PINN

5 Implications and Advantages of PIELM

PIELM offers several significant advantages for solving the biharmonic equation and potentially other high-order PDEs:

1. **Rapid Solution:** PIELM solves high-order linear PDEs without using prior data or parallel computation, with speed approximately 1000 times faster than original PINNs.
2. **Systematic Architecture Selection:** PIELM reduces the arbitrariness in neural network architecture selection by providing a clear criterion for determining the optimal number of hidden neurons.
3. **Optimization Robustness:** Being a single-layer network, PIELM avoids optimization difficulties like vanishing gradients and local minima that affect deep networks.
4. **Complex Geometry Handling:** PIELM's meshless nature makes it ideal for problems with complicated geometries where traditional mesh generation is difficult or impossible.
5. **Direct Analytical Solution:** By transforming PDEs into systems of linear equations, PIELM provides a direct analytical solution method that avoids iterative optimization procedures.

These advantages position PIELM as a promising approach for engineering applications involving high-order PDEs in complex domains, such as fluid flow simulations in intricate geometries, structural analysis of irregularly shaped components, and heat transfer problems with complex boundaries.

While PIELM is slower than mesh-based methods for regular domains, it offers a compelling alternative for problems with complex geometries where mesh generation is computationally taxing. The current formulation is limited to linear PDEs, with extension to nonlinear problems identified as a direction for future research.